UTS Insearch
UNIVERSITY OF TECHNOLOGY SYDNEY

Socket Programming Project: Web Server

Diploma of IT
Network Fundamentals INEF001

Subject Credit Points:
Subject Coordinator: Claire Pierse
Weight: 20%
DUE: Week 6

SOCKET PROGRAMMING PROJECT: WEB SERVER

TASK DESCRIPTION

In this assignment, you will learn the basics of socket programming for TCP connections in python: how to create a socket, bind it to a specific address and port, as well as send and receive a HTTP packet. You will also learn some basics of HTTP header format.

OBJECTIVES

You will develop a web server that handles one HTTP request at a time. Your web server should accept and parse the HTTP request, get the requested file from the server's file system, create an HTTP response message consisting of the requested file preceded by header lines, and then send the response directly to the client. If the requested file is not present in the server, the server should send an HTTP "404 Not Found" message back to the client.

PROJECT BRIEF

Below you will find the skeleton code for the Web server. You are to complete the skeleton code. The places where you need to fill in code are marked with #Fill in start and #Fill in end. Each place may require one or more lines of code.

```
#import socket module
from socket import *
import sys # In order to terminate the program

serverSocket = socket(AF_INET, SOCK_STREAM)
#Prepare a server socket
#Fill in start
#Fill in end

while True:
  #Establish the connection
  print('Ready to serve...')
  connectionSocket, addr = #Fill in start .....#Fill in end
  try:
    message = #Fill in start .....#Fill in end
    filename = message.split()[1]
    f = open(filename[1:])
    outputdata = #Fill in start..... #Fill in end

    #Send one HTTP header line into socket
    #Fill in start
    #Fill in end

    #Send the content of the requested file to the client
    for i in range(0, len(outputdata)):
      connectionSocket.send(outputdata[i].encode())
    connectionSocket.send("\r\n".encode())
    connectionSocket.close()

  except IOError:
    #Send response message for file not found
    #Fill in start
    #Fill in end

    #Close client socket
    #Fill in start
    #Fill in end

serverSocket.close()
sys.exit() #Terminate the program after sending the corresponding data
```

To test the project: place an HTML file (e.g., HelloWorld.html) in the same directory as the server. Run the server program. Determine the IP address of the host running the server (e.g., 128.238.251.26). From another host, open a browser and enter the corresponding URL. For example:

http://128.238.251.26:6789/HelloWorld.html

HelloWorld.html is the name of the file you placed in the server directory. Note also the use of the port number after the colon. Replace this port number with whatever port you have used in the server code. In the above example, we have used port number 6789. The browser should then display the contents of HelloWorld.html. If you omit ":6789", the browser will assume port 80, and you will get the web page from the server only if your server is listening on port 80. Next test the project by attempting to access a file that is not present on the server for example:

http://128.238.251.26:6789/Helloorld.html.. You should receive a “404 Not Found” message.

SUBMISSION

a. Completed Server Code

Submit the final Python files (.py) implementing the server functionality. Name the file using your name and student number. If you complete the extension challenge, also submit a Multithreaded Server file and a Multithreaded client file.

b. Troubleshooting Log

Throughout your Python UDP/TCP socket programming assignment, maintain a troubleshooting log. Document your progress, errors, debugging attempts, and what you learned. Include at least **6 log entries** spread over the development period, of approx. 400 words. Include in your log:

a. Error screenshots
b. Code snippets (only the portions relevant to debugging)
c. Terminal output
d. A summary of each debugging attempt
e. What you changed and why
f. Final reflection:
    i. What took the most time?
    ii. What you learned about debugging sockets

c. Video Presentation.

Create a video presentation in Canvas Studio to showcase your work effectively. The video presentation constitutes 10% of your overall 20% project grade, consider the following scenario and tips when creating the video.

Scenario: As part of the interview for an IT graduate internship, you are required to give a presentation on your socket programming project. The panel has limited technical background in this area. The key goal is not to demonstrate deep technical mastery, but to communicate your work clearly and effectively to a non-technical audience.

The video should demonstrate the following:

(a) You successfully receive and display the contents of the HelloWorld.html file in the browser.

(b) Attempting to access a non-existent file on the server results in a “404 Not Found” message.

Demonstration Tips:

- Explain what the project does, its relevance to TCP and the properties of TCP it demonstrates.
- Emphasise real-world relevance; that is, try to think of a suitable everyday analogy.
- Use visuals to support your explanation and keep the audience engaged.
- Demonstrate the key functionalities of your project and include explanations of some key code components; however, avoid excessive detail. It is not necessary to explain the code line by line.
- Avoid reading directly from a script.
- Ensure you include your image in the video – remember this is a job application.
- Ensure your voice is clear and free from background noise.
- Limit the video to 4 minutes to keep it concise and focused. If you are submitting the HD extension challenge you may submit a video of up to 7 minutes' duration

EXTENSION CHALLENGE - TO SECURE “HD” GRADE

Currently, the web server handles only one HTTP request at a time. Implement a multithreaded server that can serve multiple clients simultaneously. Using threading, first



create a main thread in which your modified server listens for clients at a fixed port. **When it receives a TCP connection request from a client, it will set up the TCP connection through another port and services the client request in a separate thread. There will be a separate TCP connection in a separate thread for each request/response pair.**

- Modify the server to be multi-threaded, so that it can handle multiple client connections concurrently.
- Implement a main server thread that listens for incoming TCP connections on a fixed port. When a client connects, the server should accept the connection and create a separate thread to handle that client's HTTP request(s).
- Each client connection must use its own TCP socket, and each socket must be handled in its own thread, allowing multiple clients to be serviced simultaneously.
- Instead of using a browser, write your own HTTP client to test your server. Your client will connect to the server using a TCP connection, send an HTTP request to the server, and display the server response as an output. You can assume that the HTTP request sent is a GET method.
- The client should take command line arguments specifying the server IP address or host name, the port at which the server is listening, and the path at which the requested object is stored at the server. The following is an input command format to run the client.

  client.py server_host server_port filename

ASSESSMENT CRITERIA

CRITERIA	WEIGHT	SLOs	PLOs
System Design, Develop and Implement Designs, Test Solutions	**25%**	SLO2	PLO2
		SLO4	PLO7 & PLO8
Develop Presentation **LLO003: Give an effective academic presentation.**	**50%**	SLO2	PLO2
		SLO4	PLO7 & PLO8
Project diary **LLO004: Write an effective academic text**	**25%**	SLO2	PLO2
		SLO4	PLO7 & PLO8

SLOs: subject learning outcomes
PLOs: program learning outcomes